

Ibex labs: change one setting at a time

Experiments on the flow in docs/ibex-flow.pdf

Item	Value
Author	Cesar Valentin
Repository	https://github.com/XxMasterepicxX/ibex-sg13g2-chip
Needs	a run through place and route, stages 1 to 4 of docs/ibex-flow.pdf
Time	about 15 minutes of Fusion Compiler per lab

Contents

How every lab works.....	3
Lab 1: Three routing layers instead of four.....	4
Lab 2: A harder target for Fusion Compiler.....	4
Lab 3: A faster real clock.....	5
Lab 4: A partial placement blockage over a hotspot.....	6
Lab 5: Wider spacing on every signal wire.....	6
Lab 6: Redundant vias.....	7
All labs side by side.....	8

How every lab works

Each lab asks one question about the chip and answers it by changing one setting. It makes its own run, adds one line to that run's `run_cfg.tcl`, builds through place and route, and compares the result with a run that has no change. Make that unchanged run once, first. Do not compare with `my_chip`: its margin rounds rewrite its reports later. Run one lab at a time, since Fusion Compiler licenses are shared.

```
bash ~/flash/chip/new_run.sh lab_base
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_base
```

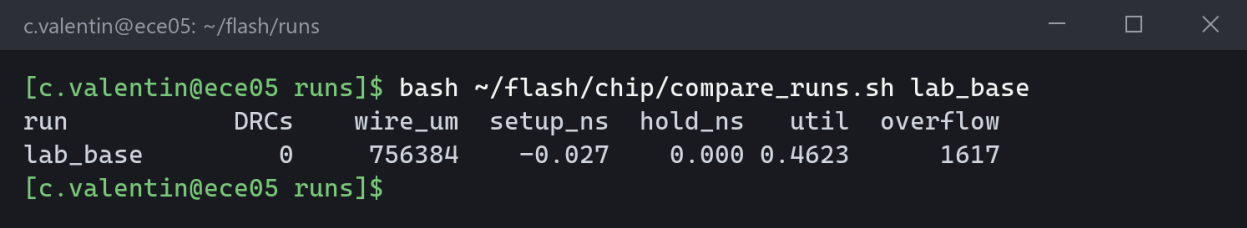
Then each lab, here the first one:

```
bash ~/flash/chip/new_run.sh lab_layers
echo "set MAX_LAYER Metal4" >> ~/flash/runs/lab_layers/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_layers
bash ~/flash/chip/compare_runs.sh lab_base lab_layers
```

`compare_runs.sh` prints one line per run:

Column	What it is
DRCs	routing rule errors after routing. Must be 0
wire_um	total length of all signal wires, in μm . Longer wires mean more delay and power
setup_ns	Fusion Compiler's worst setup slack against its own target, 10% tighter than the real clock. A small negative number is normal; signoff judges the real period
hold_ns	the worst hold slack, the same way
util	the share of the core area the cells cover
overflow	how much the first global route wanted more wiring tracks than there were, summed over the core. It shows how crowded the placement is before the router repairs it

The screenshots compare each lab with such a `lab_base`. Two runs with no change still differ a little, because the tools take slightly different paths each time: the first unchanged run, the one in the flow guide, ended at -0.107 ns of setup slack and $756,681$ μm of wire, and `lab_base` at -0.027 ns and $756,384$ μm . Read only differences larger than that as a result of the setting.



```
c.valentin@ece05: ~/flash/runs
[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base
run          DRCs    wire_um  setup_ns  hold_ns   util    overflow
lab_base      0      756384   -0.027    0.000    0.4623    1617
[c.valentin@ece05 runs]$
```

Figure 1. The unchanged run, `lab_base`, the line every lab is compared with.

Each lab below also says what else to look at. To see the layout of a lab, open its saved route step:

```
bash ~/flash/chip/open_fc.sh ~/flash/runs/lab_layers route
gui_start
```

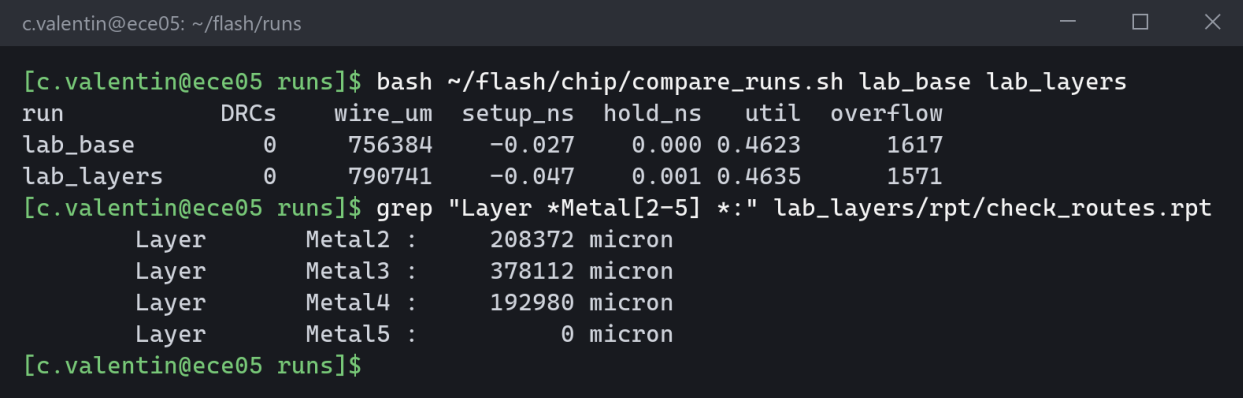
Lab 1: Three routing layers instead of four

The chip routes its signals on Metal2 to Metal5, four layers. Does it need all four? MAX_LAYER sets the top routing layer, so Metal4 leaves three.

```
bash ~/flash/chip/new_run.sh lab_layers
echo "set MAX_LAYER Metal4" >> ~/flash/runs/lab_layers/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_layers
bash ~/flash/chip/compare_runs.sh lab_base lab_layers
```

The routing report gives the wire on each layer. Metal5 should carry none:

```
grep "Layer *Metal[2-5] *:" ~/flash/runs/lab_layers/rpt/check_routes.rpt
```



```
c.valentin@ece05: ~/flash/runs

[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_layers
run          DRCs    wire_um  setup_ns  hold_ns   util  overflow
lab_base      0      756384   -0.027    0.000  0.4623   1617
lab_layers    0      790741   -0.047    0.001  0.4635   1571
[c.valentin@ece05 runs]$ grep "Layer *Metal[2-5] *:" lab_layers/rpt/check_routes.rpt
      Layer      Metal2 :      208372 micron
      Layer      Metal3 :      378112 micron
      Layer      Metal4 :      192980 micron
      Layer      Metal5 :           0 micron
[c.valentin@ece05 runs]$
```

Figure 2. lab_layers against the unchanged run.

Still 0 routing errors. Metal5 carries no wire, and the other three layers carry 4.5% more in total, 790,741 μm against 756,384 μm . The setup slack moves less than it does between two unchanged runs.

This design has room to spare: it routes on three layers. A design with more logic per area would not, which is why the number of routing layers is a real cost decision.

Lab 2: A harder target for Fusion Compiler

Fusion Compiler builds for a clock 10% faster than the 20 ns that signoff checks, to leave room. PNR_MARGIN 0.25 makes it aim for 15 ns instead of 18 ns. Does it get there, and what does it cost?

```
bash ~/flash/chip/new_run.sh lab_clock
echo "set PNR_MARGIN 0.25" >> ~/flash/runs/lab_clock/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_clock
bash ~/flash/chip/compare_runs.sh lab_base lab_clock
```

The timing report shows the target it built for, at the clock's capture edge:

```
grep -m2 "clock clk (rise edge)" ~/flash/runs/lab_clock/rpt/timing_setup_final.rpt
```

```
c.valentin@ece05: ~/flash/runs

[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_clock
run          DRCs    wire_um  setup_ns  hold_ns  util  overflow
lab_base      0      756384   -0.027    0.000  0.4623  1617
lab_clock     0      760143   -0.031    0.001  0.4682  1604
[c.valentin@ece05 runs]$ grep -m2 "clock clk (rise edge)" lab_clock/rpt/timing_setup_final.rpt
clock clk (rise edge)                0.000    0.000
clock clk (rise edge)                15.000    15.000
[c.valentin@ece05 runs]$
```

Figure 3. lab_clock against the unchanged run. Its setup slack is against 15 ns, not 18 ns.

Fusion Compiler came within 0.031 ns of 15 ns, with 0 routing errors. The wire is 0.5% longer, and the cells cover 0.468 of the core instead of 0.462.

Pushing the tool harder costs little here, so the design has timing to spare. On a design near its limit, the same setting adds buffers and larger cells, and area and power go up.

Lab 3: A faster real clock

PERIOD_NS is the clock period the chip is signed off at, 20 ns or 50 MHz. At 12 ns, 83 MHz, with the same 10% margin, Fusion Compiler aims for 10.8 ns. Where does timing break?

```
bash ~/flash/chip/new_run.sh lab_period
echo "set PERIOD_NS 12.0" >> ~/flash/runs/lab_period/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_period
bash ~/flash/chip/compare_runs.sh lab_base lab_period
```

The worst path shows where the time goes. Open the final step and ask for it:

```
bash ~/flash/chip/open_fc.sh ~/flash/runs/lab_period final
report_timing
```

```
c.valentin@ece05: ~/flash/runs

[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_period
run          DRCs    wire_um  setup_ns  hold_ns  util  overflow
lab_base      0      756384   -0.027    0.000  0.4623  1617
lab_period    0      816288   -0.036    0.002  0.4914  1161
[c.valentin@ece05 runs]$ grep -m2 "clock clk (rise edge)" lab_period/rpt/timing_setup_final.rpt
clock clk (rise edge)                0.000    0.000
clock clk (rise edge)                10.800    10.800
[c.valentin@ece05 runs]$
```

Figure 4. lab_period against the unchanged run. Its setup slack is against 10.8 ns.

It still almost closes: -0.036 ns against 10.8 ns, with 0 routing errors. To get there Fusion Compiler used larger and more cells, which now cover 0.491 of the core instead of 0.462, and 7.9% more wire.

Timing does not break at 12 ns in place and route, so the design could likely run near 80 MHz. Signoff would decide: it checks the worst wire model at every corner, where the margin is smaller. The price of speed shows up as area, 6% more cells here.

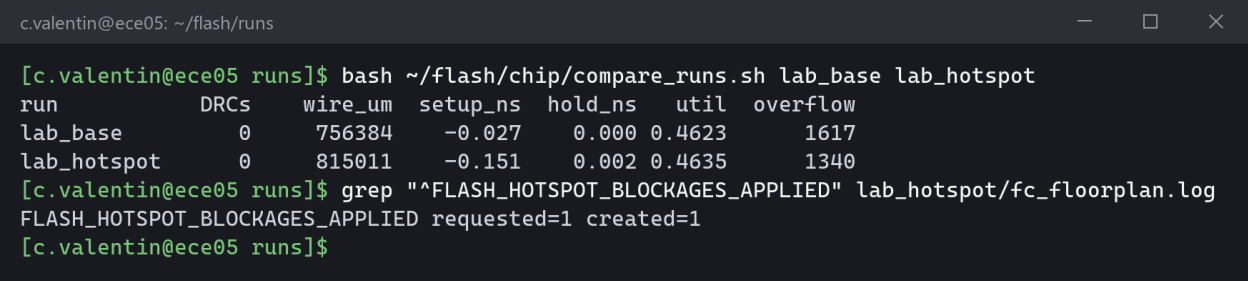
Lab 4: A partial placement blockage over a hotspot

Where cells crowd together, the wires between them crowd too, and routing errors start there. A partial placement blockage tells placement to fill a region only partly. This one allows 40% fewer cells in a box in the middle of the logic, 450 to 900 μm across and 350 to 650 μm up.

```
bash ~/flash/chip/new_run.sh lab_hotspot
echo "set HOTSPOT_BLOCKAGES {{{450 350} {900 650}} 40}" >> ~/flash/runs/lab_hotspot/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_hotspot
bash ~/flash/chip/compare_runs.sh lab_base lab_hotspot
```

The log confirms the blockage, and the placement step shows the thinner region:

```
grep FLASH_HOTSPOT_BLOCKAGES_APPLIED ~/flash/runs/lab_hotspot/fc_floorplan.log
bash ~/flash/chip/open_fc.sh ~/flash/runs/lab_hotspot place
gui_start
```



```
c.valentin@ece05: ~/flash/runs

[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_hotspot
run          DRCs    wire_um  setup_ns  hold_ns   util   overflow
lab_base      0      756384   -0.027    0.000  0.4623   1617
lab_hotspot   0      815011   -0.151    0.002  0.4635   1340
[c.valentin@ece05 runs]$ grep "^FLASH_HOTSPOT_BLOCKAGES_APPLIED" lab_hotspot/fc_floorplan.log
FLASH_HOTSPOT_BLOCKAGES_APPLIED requested=1 created=1
[c.valentin@ece05 runs]$
```

Figure 5. *lab_hotspot* against the unchanged run.

The blockage was applied, and the first global route's overflow fell 17%, from 1,617 to 1,340. The price: 7.7% more wire, 815,011 μm , and the setup slack fell to -0.151 ns, more than run-to-run variation, because related cells moved farther apart.

A blockage trades crowding for distance. On a design with routing errors in a hotspot it can remove them. This chip already routes with 0 errors, so here it only costs wire and timing. Put a blockage where the congestion map shows a hotspot, not everywhere.

Lab 5: Wider spacing on every signal wire

A routing rule can keep wires farther apart than the minimum, which helps with rules the router does not see and with crosstalk. This lab puts double spacing on every signal net.

```
bash ~/flash/chip/new_run.sh lab_ndr
echo "set RUN_NDR {spacing 2 width 1}" >> ~/flash/runs/lab_ndr/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_ndr
bash ~/flash/chip/compare_runs.sh lab_base lab_ndr
```

The log counts the nets that got the rule:

```
grep FLASH_NDR ~/flash/runs/lab_ndr/fc_floorplan.log
```

```
c.valentin@ece05: ~/flash/runs

[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_ndr
run          DRCs    wire_um  setup_ns  hold_ns   util   overflow
lab_base      0      756384   -0.027    0.000  0.4623    1617
lab_ndr       0     1189896  -0.092   -0.003  0.4622   112769
[c.valentin@ece05 runs]$ grep "^FLASH_NDR" lab_ndr/fc_floorplan.log
FLASH_NDR spacing 2 width 1 on 13096 signal nets
[c.valentin@ece05 runs]$
```

Figure 6. lab_ndr against the unchanged run.

Still 0 routing errors, but 57% more wire, 1,189,896 μm , and the first global route's overflow jumped from 1,617 to 112,769: double spacing leaves far fewer tracks. Fast hold slipped just under 0 at three endpoints.

Wide spacing is a tool for the few nets that need it, such as a clock or a long bus, not for all of them.

Lab 6: Redundant vias

A via joins a wire on one layer to the next. A single via that fails opens the net. REDUNDANT_VIAS adds a second via wherever there is room, after routing.

```
bash ~/flash/chip/new_run.sh lab_vias
echo "set REDUNDANT_VIAS 1" >> ~/flash/runs/lab_vias/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_vias
bash ~/flash/chip/compare_runs.sh lab_base lab_vias
```

The log has Fusion Compiler's own report of what it did:

```
grep -h "ZRT-829\|Total double via conversion rate" ~/flash/runs/lab_vias/fc_floorplan.log
```

```
c.valentin@ece05: ~/flash/runs

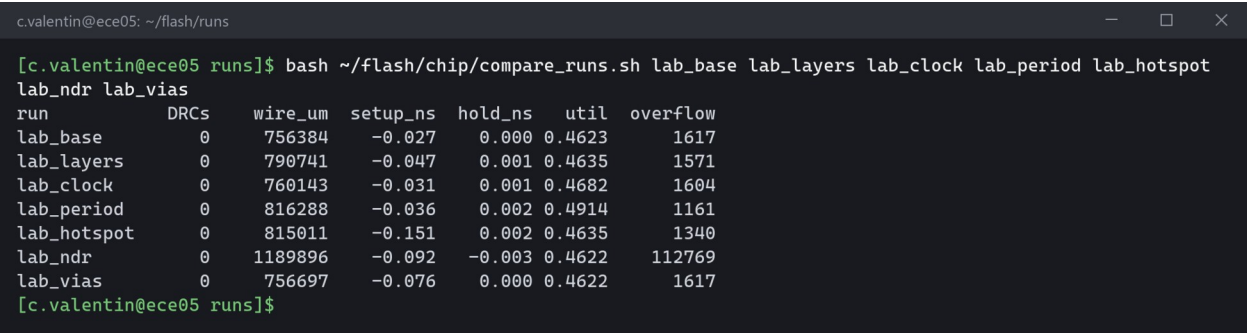
[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_vias
run          DRCs    wire_um  setup_ns  hold_ns   util   overflow
lab_base      0      756384   -0.027    0.000  0.4623    1617
lab_vias      0      756697   -0.076    0.000  0.4622    1617
[c.valentin@ece05 runs]$ grep -h "ZRT-829\|Total double via conversion rate" lab_vias/fc_floorplan.log
Total double via conversion rate = 0.00% (0 / 8582 vias)
Total double via conversion rate = 0.00% (0 / 123836 vias)
Total double via conversion rate = 0.00% (0 / 123811 vias)
Total double via conversion rate = 0.00% (0 / 123811 vias)
Total double via conversion rate = 0.00% (0 / 138095 vias)
Total double via conversion rate = 0.00% (0 / 138306 vias)
Total double via conversion rate = 0.00% (0 / 138306 vias)
Warning: Redundant via insertion will be skipped because a via mapping was not provided. (ZRT-829)
[c.valentin@ece05 runs]$
```

Figure 7. lab_vias against the unchanged run.

Nothing changed. Fusion Compiler skipped the step with warning ZRT-829: no via mapping tells it which double vias IHP allows, and its own report gives a conversion rate of 0.00% of 138,306 vias. The results match the unchanged run within run-to-run variation.

A setting can be accepted and still do nothing, so always check the tool's own report of what it did. Making this one work needs a via mapping for IHP's vias, `add_via_mapping` in Fusion Compiler, which this flow does not have yet.

All labs side by side



```
c.valentin@ece05: ~/flash/runs
[c.valentin@ece05 runs]$ bash ~/flash/chip/compare_runs.sh lab_base lab_layers lab_clock lab_period lab_hotspot lab_ndr lab_vias
run      DRCs    wire_um  setup_ns  hold_ns   util    overflow
lab_base    0      756384   -0.027    0.000  0.4623    1617
lab_layers  0      790741   -0.047    0.001  0.4635    1571
lab_clock   0      760143   -0.031    0.001  0.4682    1604
lab_period  0      816288   -0.036    0.002  0.4914    1161
lab_hotspot 0      815011   -0.151    0.002  0.4635    1340
lab_ndr     0     1189896   -0.092   -0.003  0.4622   112769
lab_vias    0      756697   -0.076    0.000  0.4622    1617
[c.valentin@ece05 runs]$
```

Figure 8. Every lab and the unchanged run, from `compare_runs.sh`.

Other settings to try the same way: `CORE_SIZE`, the core size in μm , which must stay a whole number of $3.78 \mu\text{m}$ rows; `MACRO_HALO`, the keepout around the memories; `PNR_MARGIN` between 0 and 0.25; `MAX_LAYER` Metal3. `chip/run_cfg.tcl` lists the settings this chip uses, and the first lines of `flow/fc_flow.tcl` list the rest.